PORTFOLIO ARCHITECTURE WALKTHROUGH

# AWS Billing Notification Automation

A configurable, event-driven system for Transfer Billing notifications

Joseph Abramov · Cornell CS · May 2026

EventBridge | Lambda | Config-driven | Idempotent

# Why this project mattered

Transfer Billing creates high-stakes customer notifications across a billing lifecycle.

**Billing responsibility moves**

Account A can transfer billing responsibility to Account B, which creates lifecycle events.

→

**Customers need updates**

Examples: initiated, scheduled, termination approaching, error states.

→

**Mistakes are costly**

Incorrect or duplicated emails affect trust, compliance, and billing clarity.

Design implication

## The system needed to be reliable enough for billing communications while still letting the team add new notification types quickly.

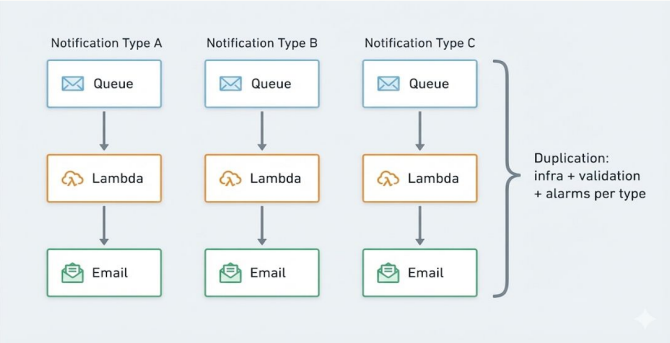
# From many pipelines to one configurable system

The legacy approach was hard to extend; the target system made new email types configuration-driven.

LEGACY APPROACH

**Separate pipeline per email type**

Each new notification required new service wiring, deployment steps, and testing paths. Growth meant duplicated infrastructure and slower delivery.




→

TARGET OUTCOME

**One pipeline, many notification types**

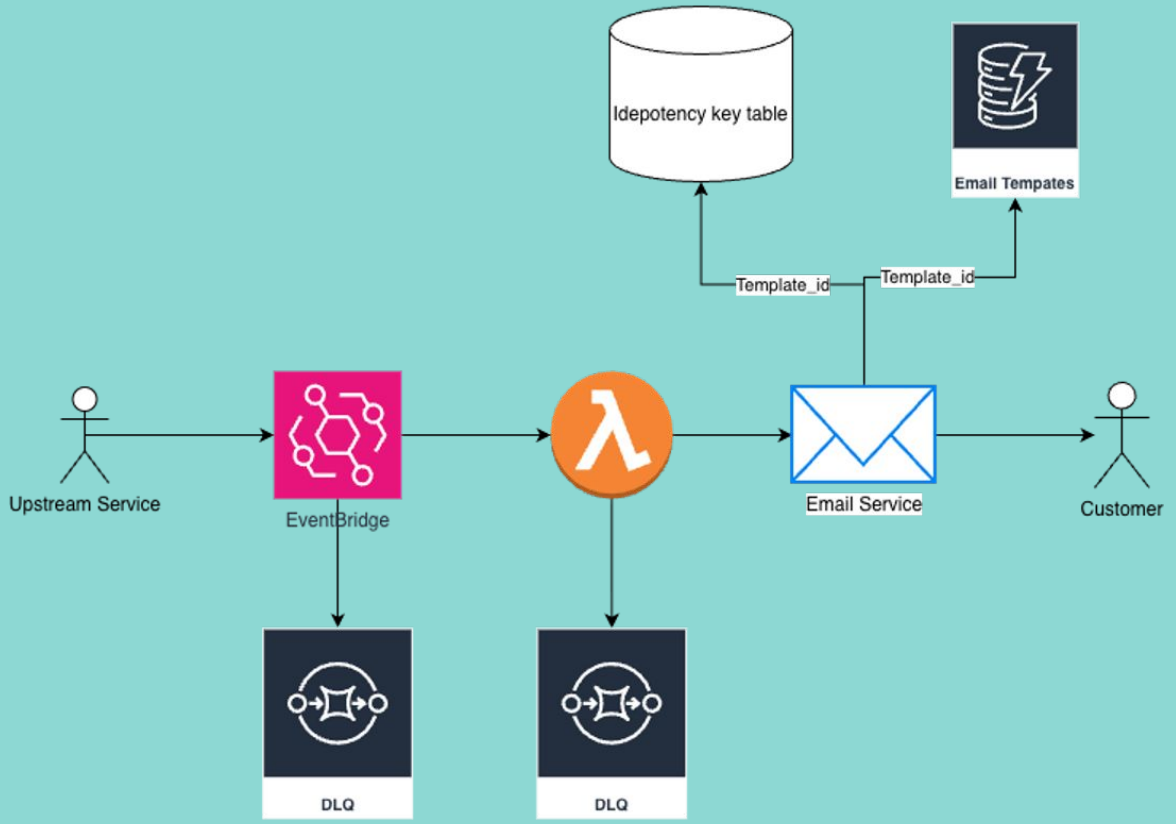
A JSON config defines valid events, required fields, and template IDs so new notifications can be added without rebuilding the system.

- Add new email types fast
- Reject invalid events before they reach core logic
- Prevent duplicate logical notifications during retries

# ASYNC EVENT-DRIVEN ARCHITECTURE

Why Async Matters:

- Upstream publishes event and continues without waiting
- Lambda processes when ready
- Emails don’t need immediate response so async is ideal
- Perfect for best effort delivery



# Async event-driven architecture

The system decoupled producers from email delivery and used each AWS component for a narrow purpose.

**Upstream services**

Publish billing lifecycle event

→

**EventBridge**

Route, filter, enrich, DLQ invalid events

→

**Lambda**

Validate safety net + call email service

→

**Internal email service**

Render, dedupe, retry, deliver

## Why async?

- The publisher does not block while email work happens
- Bursty notification traffic can be absorbed without slowing upstream workflows
- Email delivery is important, but it does not require a synchronous user response

# Why EventBridge + Lambda

The architecture kept routing and orchestration separate instead of forcing one component to do everything.

**EventBridge handled event routing**

- Decouples producers from email delivery
- Filters events by type
- Enriches valid events with template_id
- Sends invalid events to a DLQ early

**Lambda stayed thin by design**

- Runs safety validation
- Calls the internal email service
- Avoids owning template rendering or delivery retries
- Keeps operational overhead low

**Portfolio takeaway**

**The main engineering decision was not “use AWS services.” It was deciding which component should own routing, validation, orchestration, and delivery reliability.**

# JSON config made new notifications easier to add

Instead of changing code for each email type, behavior moved into a structured configuration.

**Config contained**

- event_types whitelist
- required_params for template fields
- template_id for rendering

→

**EventBridge used it to**

- validate event type
- enrich valid events
- route failures to DLQ

→

**Lambda received**

A cleaner event with known fields and the correct template_id already attached.

**Why this mattered**

## Config-driven design reduced deployment friction and made the system easier for the team to extend after handoff.

# Defining “same notification” was the hard part

Retries and legitimate repeated emails make deduplication a design problem, not just a hash function.

**Why timestamps failed**

Upstream retries can change metadata. Event time is not stable enough to identify a logical notification.

**Stable uniqueness signal**

I recommended a scheduled_date field so retries and replays refer to the same intended billing notice.

**Key design**

Normalize email + relationship + notification type + scheduled_date. Store keys with a 14-day TTL.

**Design rule**

## Customer gets one logical email per relationship, per day, per notification type — even if upstream retries.

# Testing strategy

The testing plan focused on validation edges, end-to-end behavior, and duplicate prevention.

**Unit tests**

Validation logic and edge cases; target coverage above 90%.

**Integration test**

End-to-end events triggered emails to test recipients.

**Idempotency test**

Same logical event sent twice; no duplicate emails observed.

**Capacity**

Designed for 40+ TPS.

**Result**

**The system caught invalid events early, sent expected emails correctly, and avoided duplicates during retry simulation.**

# Impact and handoff

The project turned a slow, repetitive workflow into a reusable notification platform.

**Weeks** → **4 hours**

new email deployment time

**40+ TPS**

Capacity target for billing-cycle traffic

**0 duplicates**

Observed during duplicate-injection testing

**1 pipeline**

Replaced 6+ proposed separate paths

**Handoff outcome**

Interns could not push the production launch directly, so I prepared detailed handoff and runbook documentation. The project was approved and handed to the team for continuation.

# What I learned

The project taught me how quickly implementation improves once the system contract is clear.

**What worked**

- Requirements first
- Async architecture for notification flow
- Config-driven design
- Stable idempotency contract

**What I would change**

- Start simpler on validation
- Think about email tracking earlier
- Separate MVP from nice-to-have features faster

**Professional takeaway**

Distributed systems require clear contracts, careful failure handling, and simple extension paths.

**Why this belongs on my portfolio**

## This walkthrough shows the engineering judgment behind the project, not just the technologies listed on my resume.